IFB104 Practice Quiz: Building IT Systems

Question 1

Selection statements allow a program to choose between alternative actions depending on the value of a Boolean expression. A common source of bugs is confusing a sequence of separate `if` statements with a single `if-elif-else` chain, because the two behave very differently. Consider the following code, which is intended to convert a numeric score into a grade.

```
score = 90

if score >= 50:
    grade = 'Pass'
if score >= 70:
    grade = 'Credit'
elif score >= 85:
    grade = 'Distinction'
else:
    grade = 'Fail'

print(grade)
```

What is printed when this code runs?

A. `Pass`

B. `Distinction`

C. `Credit`

D. `Fail`

E. `Pass` then `Credit` on separate lines

Question 2 (multiple answer)

Boolean expressions can combine several conditions using the operators `and`, `or` and `not`. When these operators are mixed in a single expression without brackets, Python evaluates them in a fixed order of precedence, which may not be the order you would read them in. Consider the following function, which suggests an activity based on the weather.

```
def choose_activity(temperature, raining):
    if temperature > 30 or raining and temperature > 20:
        return 'Swim'
    elif not raining:
        return 'Walk'
    else:
        return 'Read'
```

Which of the following function calls return `'Swim'`? Select all that apply.

A. `choose_activity(25, False)`

B. `choose_activity(35, False)`

C. `choose_activity(15, True)`

D. `choose_activity(25, True)`

E. `choose_activity(20, True)`

Question 3

Functions allow us to package up a piece of code and reuse it with different inputs, called arguments. When a variable is passed to a function, it is important to understand whether changes made to the parameter inside the function affect the original variable outside it. Consider the following code, which is intended to add a bonus to a student's mark.

```
def add_bonus(score):
    score = score + 10
    return score

mark = 50
add_bonus(mark)
print(mark)
```

What is printed when this code runs?

A. `60`

B. `50`

C. `None`

D. `10`

E. `UnboundLocalError`

Question 4

Unlike numbers and strings, lists are mutable, which means their contents can be changed after they have been created. When a list is passed to a function, the function's parameter refers to the same list object as the caller's variable. Consider the following two functions, which both attempt to change a shopping list.

```
def add_item(basket, item):
    basket.append(item)

def replace_basket(basket):
    basket = ['empty']

shopping = ['milk']
add_item(shopping, 'eggs')
replace_basket(shopping)
print(shopping)
```

What is printed when this code runs?

A. `['milk']`

B. `['empty']`

C. `['empty', 'eggs']`

D. `['milk', 'eggs']`

E. `['eggs']`

Question 5

In Python, an assignment statement such as `a = b` does not copy a value. Instead, it makes the name `a` refer to the same object as `b`. This distinction is especially important when working with mutable objects such as lists. Consider the following code segment.

```
original = [1, 2, 3]
alias = original
copy = original[:]

alias.append(4)
copy.append(5)

print(original, copy)
```

What is printed when this code runs?

A. `[1, 2, 3, 4] [1, 2, 3, 5]`

B. `[1, 2, 3, 4, 5] [1, 2, 3, 4, 5]`

C. `[1, 2, 3] [1, 2, 3, 5]`

D. `[1, 2, 3, 4] [1, 2, 3, 4, 5]`

E. `[1, 2, 3, 4, 5] [1, 2, 3, 5]`

Question 6

Strings and lists are both sequences, so their elements can be accessed by position using indexing and slicing. Index positions start at zero, and Python also allows negative indices, which count backwards from the end of the sequence. Consider the following code, where variable `city` holds the name of a city.

```
city = 'BRISBANE'
print(city[2:5], city[-3:])
```

What is printed when this code runs?

A. `RIS NE`

B. `ISBA ANE`

C. `ISB ANE`

D. `RISB ENA`

E. `ISB NAB`

Question 7

Loops are frequently used to process each element of a list in turn. One way of doing so is to loop over the index positions of the list and use each position to access an element. When using this approach, the range of positions produced by the loop must exactly match the list's valid indices. Consider the following code segment, which is intended to print every number in the list on a single line.

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]

for position in range(1, len(numbers) + 1):
    print(numbers[position], end=' ')
```

What is the result of running this code?

A. `1 2 3 4 5 6 7 8 9` is printed and the program ends normally

B. `1 2 3 4 5 6 7 8 9` is printed, then an `IndexError` is raised

C. `2 3 4 5 6 7 8 9` is printed and the program ends normally

D. An `IndexError` is raised before anything is printed

E. `2 3 4 5 6 7 8 9` is printed, then an `IndexError` is raised

Question 8

A divisor of a whole number is a number that divides into it exactly, with no remainder. For example, the divisors of 6 are 1, 2, 3 and 6. Python's remainder operator `%` is a convenient way of testing for divisors, because `number % candidate` equals zero exactly when `candidate` divides `number`. Consider the following function, which is intended to count how many divisors a positive whole number has.

```
def count_divisors(number):
    count = 0
    for candidate in range(number + 1):
        if number % candidate == 0:
            count = count + 1
    return count

print(count_divisors(6))
```

What is the result of running this code?

A. Nothing is printed; a `ZeroDivisionError` is raised

B. `4`

C. `3`

D. `5`

E. `0`

Question 9

When a Python program encounters a problem that it cannot handle, it raises an exception. The type of the exception tells the programmer what kind of problem occurred, which is the first step in finding and fixing a bug. Consider the following four lines of code, each of which is run on its own.

```
'3' + 4                 # Line 1
{'apple': 3}['banana']  # Line 2
[10, 20][2]             # Line 3
int('four')             # Line 4
```

Which exception does each line raise, in order from Line 1 to Line 4?

A. `ValueError`, `IndexError`, `KeyError`, `TypeError`

B. `TypeError`, `IndexError`, `KeyError`, `ValueError`

C. `SyntaxError`, `KeyError`, `IndexError`, `TypeError`

D. `TypeError`, `KeyError`, `IndexError`, `ValueError`

E. `TypeError`, `KeyError`, `IndexError`, `TypeError`

Question 10

In Assignment 1 you processed data stored as a list of lists, where each inner list represents one record and named constants are used to identify the position of each field. Consider the following code, which processes the results for a small group of students. A student passes only if they score at least 50 in both the exam and the assignment.

```
results = [['Ana', 72, 85],
           ['Ben', 45, 38],
           ['Cy', 90, 49],
           ['Dee', 50, 50]]
NAME, EXAM, ASSIGNMENT = 0, 1, 2

passed = []
for student in results:
    if student[EXAM] >= 50 and student[ASSIGNMENT] >= 50:
        passed.append(student[NAME])

print(passed)
```

What is printed when this code runs?

A. `['Ana', 'Cy', 'Dee']`

B. `['Ana', 'Dee']`

C. `['Ana']`

D. `['Ben', 'Cy']`

E. `[['Ana', 72, 85], ['Dee', 50, 50]]`

Question 11

Errors in Python programs fall into two broad groups: those that Python detects before the program starts running, and those that only appear while the program is running. Knowing which group an error belongs to helps explain why some output may appear before a program fails. Consider the following program, which contains a small typing mistake.

```
def total_price(prices):
    total = 0
    for price in prices:
        total = total + price
    return totl

print('Start')
result = total_price([2, 3])
print(result)
```

What happens when this program runs?

A. A `SyntaxError` is reported and nothing is printed

B. A `NameError` is raised before `Start` is printed

C. `Start` is printed, then a `NameError` is raised

D. `Start` is printed, then `5`

E. `Start` is printed, then `None`

Question 12

Graphical user interfaces are built from widgets, and PySide6 provides a different widget for each kind of user interaction. Choosing the right widget makes an interface easier to use and the code behind it simpler to write. Suppose you are building a sign-up window that needs:

1. a box to type the user's name (one line),
2. an on/off option to subscribe to the newsletter,
3. a choice of exactly one plan (Basic, Pro or Team), with all three options visible at once,
4. a button to submit the form.

Which widgets best match items 1 to 4, in order?

A. `QTextEdit`, `QRadioButton`, `QComboBox`, `QLabel`

B. `QLineEdit`, `QRadioButton`, `QCheckBox`, `QPushButton`

C. `QLabel`, `QCheckBox`, `QListWidget`, `QPushButton`

D. `QLineEdit`, `QCheckBox`, `QRadioButton`, `QPushButton`

E. `QLineEdit`, `QPushButton`, `QComboBox`, `QLabel`

Question 13

Rather than placing widgets at fixed screen coordinates, Qt programs use layout managers to arrange widgets automatically. A `QVBoxLayout` arranges its contents vertically and a `QHBoxLayout` arranges them horizontally, and layouts can be nested inside each other to build more complex windows. Consider the following code, which builds the window for a suburb search tool. Assume that `window` is a `QWidget`.

```
main_layout = QVBoxLayout()

search_row = QHBoxLayout()
search_row.addWidget(QLabel('Suburb:'))
search_row.addWidget(QComboBox())

main_layout.addLayout(search_row)
main_layout.addWidget(QPushButton('Search'))
main_layout.addWidget(QLabel('Results'))

window.setLayout(main_layout)
```

Which sketch best shows the resulting window?

A.

```
+-----------------------------+
| Suburb:  [ combo box   v ]  |
| [         Search          ] |
| Results                     |
+-----------------------------+
```

B.

```
+-----------------------------------------------+
| Suburb: [ combo v ] [ Search ] Results        |
+-----------------------------------------------+
```

C.

```
+-----------------------------+
| Suburb:                     |
| [ combo box            v ]  |
| [         Search          ] |
| Results                     |
+-----------------------------+
```

D.

```
+-----------------------------+
| [         Search          ] |
| Results                     |
| Suburb:  [ combo box   v ]  |
+-----------------------------+
```

E.

```
+-----------------------------+
| Suburb:      | [ Search ]   |
```

```
| [ combo v ]  | Results      |
+------------------------------+
```

Question 14

GUI programs are event-driven: rather than running from top to bottom and finishing, they wait for the user to do something and then respond. In Qt this is achieved by connecting a widget's signal, such as a button's `clicked` signal, to a function known as a slot, which Qt calls whenever the signal is emitted. Consider the following code.

```
def show_message():
    print('Button clicked!')

button = QPushButton('Click me')
button.clicked.connect(show_message())
```

What happens when the last line runs?

A. Nothing happens yet; `Button clicked!` is printed each time the user clicks the button

B. `Button clicked!` is printed once straight away, then `connect` raises a `TypeError` because it was given `None`

C. Nothing is ever printed, but no error occurs

D. A `SyntaxError` is reported because `connect` needs a string

E. `Button clicked!` is printed twice on every click

Question 15

Data entered by the user into a GUI usually needs to be converted to an appropriate type before it can be used in calculations. Forgetting to do so does not always cause an error, which can make the resulting bug hard to notice. Consider the following slot function from a simple shop program. The user types `4` into `quantity_input` (a `QLineEdit`) and clicks a button connected to this function.

```
def show_total():
    quantity = quantity_input.text()
    price_per_item = 2
    total = quantity * price_per_item
    result_label.setText('Total: ' + str(total))
```

What does `result_label` display?

A. `Total: 8`

B. Nothing; a `TypeError` is raised

C. `Total: 4`

D. Nothing; a `ValueError` is raised

E. `Total: 44`

Question 16

Regular expressions describe patterns of text and are commonly used to find or validate structured tokens such as identification numbers, dates and phone numbers. Consider the following scenario. A QUT student ID consists of the lowercase letter `n` followed by exactly eight digits, for example `n12345678`, and must appear as a complete token, not embedded in other characters.

Which of the following regex patterns correctly matches a QUT student ID?

A. `n\d+`

B. `\bn\d{8}`

C. `\bn\d{8}\b`

D. `[n]\d{7}\b`

E. `\bN\d{8}\b`

Question 17

Python's `re` module provides several functions for working with regular expressions. The `findall` function searches a whole string for every non-overlapping match of a pattern and returns all of the matches together. Consider the following code.

```
import re

print(re.findall(r"\d+", "Room 12, level 3, 2026"))
```

What is printed when this code runs?

A. `[12, 3, 2026]`

B. `['1', '2', '3', '2', '0', '2', '6']`

C. `'12'`

D. `['12', '3', '2026']`

E. `['Room 12', 'level 3', '2026']`

Question 18 (multiple answer)

When validating user input, it is important that the entire input matches the expected format, not just part of it. Suppose a product code must consist of exactly two uppercase letters followed by exactly three digits, and is checked using the pattern below with `re.fullmatch(pattern, text)`.

```
import re

pattern = r"[A-Z]{2}\d{3}"
```

For which of the following values of `text` does `re.fullmatch` return a match? Select all that apply.

A. `'AB123'`

B. `'ab123'`

C. `'AB1234'`

D. `'XY000'`

E. `'A B123'`

Question 19* (multiple answer)

A recursive function is a function that calls itself. Recursion can be used instead of a loop to repeat an action, provided that the function has a base case that eventually stops the repetition. Where the `print` statement is placed relative to the recursive call also affects the order of the output. Without using any loop, which of the following programs print the numbers 1 to 5 in ascending order, one per line? Select all that apply.

A.

```
def count_up(number):
    if number > 5:
        return
    print(number)
    count_up(number + 1)

count_up(1)
```

B.

```
def count_up(number):
    if number > 5:
        return
    count_up(number + 1)
    print(number)

count_up(1)
```

C.

```
def count_up(number):
    print(number)
    count_up(number + 1)

count_up(1)
```

D.

```
def count_up(number):
    if number == 0:
        return
    count_up(number - 1)
    print(number)

count_up(5)
```

E.

```
def count_up(number):
    if number < 5:
        print(number)
        count_up(number + 1)

count_up(1)
```

Question 20

The IFB104 Code Presentation Guide was provided to improve the quality of your code on the two major assignments. It covers layout, the choice of identifier names, commenting and the avoidance of magic numbers. Each of the following code snippets converts a number of minutes into seconds. Which one best follows the guide?

A.

```
for i in range(len(names)):
    print(names[i])
```

B.

```
SECONDS_PER_MINUTE = 60
total_seconds = minutes * SECONDS_PER_MINUTE
```

C.

```
x = m*60 # multiply m by 60
```

D.

```
def Calc(m): return m*60
```

E.

```
totSecs = mins * 60 # calc secs
```

Question 21*

Lists can be used to store previously calculated results so that each new value can be built from earlier ones. A classic example is the Fibonacci sequence `0, 1, 1, 2, 3, 5, 8, 13, ...`, in which each term is the sum of the two terms before it. The function below is meant to return term `n` of the sequence, counting from 0, so `fibonacci(6)` should return `8`.

```
def fibonacci(n):
    fibo = [0] * n
    fibo[0] = 0
    fibo[1] = 1
    for index in range(2, n + 1):
        fibo[index] = fibo[index - 1] + fibo[index - 2]
    return fibo[n]

print(fibonacci(6))
```

What happens when this code runs, and what is the best fix?

A. It prints `8`; no fix is needed

B. It prints `5`; change the return statement to `return fibo[n + 1]`

C. It raises an `IndexError`; change the first line to `fibo = [0] * (n + 1)`

D. It raises an `IndexError`; change the loop to `range(2, n)` and leave everything else as it is

E. It loops forever; add a `break` inside the loop
